

# Lesson Plan — Submitting AI Jobs to the Anvil Supercomputer

**Workshop:** AI for All: Practical and Accessible AI for Research and Education **Date/venue:** Tuesday, September 15, 2026 — The Mill at MSU, Starkville, MS **Slot:** Beginner Track 1, 1:00–2:30 PM CT (90 minutes) **Instructor:** Juanjo Garcia Mesa (ASU Research Technology Office) **Audience:** Beginners — faculty, students, staff from Mississippi research institutions; no prior HPC experience assumed. Some will have just attended the morning panel on Mississippi research computing infrastructure and the “Leveraging National Level AI and HPC Resources” talk, so open by connecting to those.

## Learning objectives

By the end of the session, attendees will be able to:

1. **LO1 — Explain** the login-node vs. compute-node model and why jobs are submitted to a queue instead of run interactively.
2. **LO2 — Log in** to Anvil via SSH keys or Open OnDemand.
3. **LO3 — Submit** a batch job with `sbatch` using a correct submission script (account, partition, time, resources).
4. **LO4 — Monitor and manage** jobs: `squeue`, `sacct`, `scancel`, read output files.
5. **LO5 — Escalate** to an AI workload: run a small ML training job on CPU, then a GPU job on the `gpu-debug/gpu` partitions, requesting GPUs with `--gres`.
6. **LO6 — Know where to go next:** ACCESS/NAIRR allocations, docs, and help channels.

## Prerequisites (attendees)

- Laptop with SSH client (Terminal on macOS/Linux; PowerShell on Windows 10+ works)
- An Anvil account (ACCESS ID -> allocation added; NAIRR allocations are automatic). *Instructors: see instructor-notes.md — pre-provisioned accounts are the #1 success factor for this session.*
- The lab files (shared via short URL / QR code at the session, or `git clone` / `scp`).

## Materials

| File                             | Purpose                                                                      |
|----------------------------------|------------------------------------------------------------------------------|
| <code>slides-outline.md</code>   | ~18 slides supporting the framing and live demos                             |
| <code>handout.md</code>          | Attendee lab guide: commands, expected output, troubleshooting table         |
| <code>instructor-notes.md</code> | Pre-session checklist, timing, fallbacks                                     |
| <code>labs/</code>               | 4 self-contained labs (each: a Python script + an <code>.sbatch</code> file) |

## Session structure (15 min framing -> 60 min hands-on -> 15 min wrap-up)

### Part 0 — Pre-session (T-minus, instructor)

Done before the workshop: accounts provisioned, lab files staged in a shared location (`~<workshop-user>/ai-workshop-submit` on Anvil, plus a download URL), GPU partition availability checked, `module spider` output captured for the exact PyTorch/ container stack used in Lab 3.

### Part 1 — Framing (1:00–1:15, 15 min)

| # | Slide                                          | Time  | Talking points                                                                                                                                                                                                                                                                                                                   |
|---|------------------------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | Title + your intro                             | 1 min | One-slide bio; you're an RSE at ASU, NAIRR Pilot facilitator/mentor — connects to the morning's "Leveraging National Level AI and HPC Resources" talk.                                                                                                                                                                           |
| 2 | Why a supercomputer for AI?                    | 3 min | A single A100/H100 vs. your laptop; training/inference scale. Anvil by the numbers: 1,000+ nodes, 128 cores/node AMD EPYC, 16x A100 nodes, 21x H100 nodes, 5.3 PF.                                                                                                                                                               |
| 3 | What is a scheduler & why queues?              | 2 min | Shared resource, fair share; login node vs. compute node; <i>running on the login node is against Anvil policy</i> — say this twice.                                                                                                                                                                                             |
| 4 | Meet Anvil's queues (partitions)               | 3 min | The real table (from <code>showpartitions</code> ): <code>debug</code> , <code>gpu-debug</code> , <code>shared</code> (default), <code>wholenode</code> , <code>wide</code> , <code>highmem</code> , <code>gpu</code> (A100), <code>ai</code> (H100). Emphasize: <code>debug</code> queues are free/short — perfect for a class. |
| 5 | Anatomy of a job                               | 2 min | Request -> queue -> run -> output. Resources = cores, memory, walltime, GPUs ( <code>--gres</code> ).                                                                                                                                                                                                                            |
| 6 | The <code>#SBATCH</code> script = your request | 2 min | Walk the annotated <code>hello.sbatch</code> line by line; <b>mandatory fields on Anvil: <code>-A account</code> and <code>-p partition</code></b> — a missing account is the #1 first-submission error.                                                                                                                         |
| 7 | What we'll do today (roadmap)                  | 2 min | 4 labs: <code>hello</code> -> digits (CPU AI) -> GPU PyTorch -> job array (stretch). Everyone leaves having run a job on a national supercomputer.                                                                                                                                                                               |

*Interactive beat (slide 4): ask "who has run anything on a cluster before?" — calibrates pace.*

## Part 2 — Hands-on labs (1:15–2:15, 60 min)

Labs are check-pointed: no one falls more than one lab behind. Each lab ends with a **visible success signal** (output on screen) before moving on.

**Lab 1 — Connect and look around (12 min, 1:15–1:27) Goal:** LO2, and vocabulary for everything after.

1. SSH to `anvil.rcac.purdue.edu` (or open OnDemand in a browser — the zero-terminal fallback for anyone with SSH trouble; OnDemand has a Shell app and job submission GUI).
2. `mybalance` — see your allocation account string (the `-A` value).
3. `showpartitions` — find the queues; identify `debug` and `gpu-debug`.
4. `sfeatures` — see node types: `a[000-999]` CPU, `g[000-015]` A100 `gpu:4`, `h[000-020]` H100 `gpu:4`.

**Checkpoint:** everyone has run `showpartitions` and can name their account string. *Fallback:* OnDemand file browser view of the same info; pair up anyone stuck.

**Lab 2 — First job: hello, Anvil (12 min, 1:27–1:39) Goal:** LO3. Submit `labs/hello.sbatch` (CPU, debug partition, 10 min walltime).

1. `cd` to the lab directory; open `hello.sbatch` — find the three lines that matter: `-A`, `-p`, `-t`.
2. Edit in the account string from Lab 1.
3. `sbatch hello.sbatch ->` job ID.
4. `squeue -u $USER ->` watch it go `PENDING -> RUNNING -> gone`.
5. `cat slurm-<jobid>.out ->` “Hello from node aXXX” + Slurm environment report.

**Teaching moment:** the output file records `SLURM_JOB_NODELIST`, `SLURM_JOB_PARTITION`, `SLURM_SUBMIT_DIR` — the job ran on a *different machine* than where they typed.

**Checkpoint:** every attendee has a `slurm-*.out` with their own job ID. *Common errors:* missing `-A` (“invalid account specified”), typos in partition (`sbatch: error: Invalid partition name`).

**Lab 3 — A real AI job on CPU: scikit-learn digits (18 min, 1:39–1:57) Goal:** LO5a. Submit `labs/train-digits-cpu.sbatch` — trains a digits classifier (scikit-learn, built into Anaconda module; dataset ships with the library, **no download needed** — safe for the cluster’s egress and fast in a class).

1. Open the two files: `train_digits.py` (short, commented — classifier train + test accuracy + a saved metrics file) and the `.sbatch` (module load `conda`, more cores via `--cpus-per-task=16`, `--mem=16G`).
2. Submit to `debug` (2-hr limit fits; runs in ~1 min on 16 cores).
3. While it runs: **live demo at the front** — instructor submits the same job, walks `squeue`, `sacct -j <id>`, opens the output as it lands.
4. Retrieve results: `cat` the accuracy line; `ls` the metrics JSON.

**Teaching moment:** show how the *same* request scales — change `--cpus-per-task` and discuss what the job cost in SUs. Connect back to the morning talk: this is exactly how a national allocation is spent.

**Checkpoint:** everyone sees a test-accuracy number they produced on a supercomputer. *Fallback:* if `scikit-learn` is unavailable in the default `conda` module, instructor runs `module spider scikit-learn` output captured pre-session; `pip-in-venv` fallback script provided.

**Lab 4 — GPU job: PyTorch on the A100s (15 min, 1:57–2:12) Goal:** LO5b. Submit `labs/train-gpu.sbatch` — small PyTorch model (MNIST-style conv net or equivalent small dataset embedded/generated in the script, no download), partition `gpu-debug` (30-min limit, 2 GPUs — designed for exactly this).

1. Open the `.sbatch`: the new lines are `--gres=gpu:1` and `--partition=gpu-debug`.
2. Submit; `squeue ->` node name starts with `g` (A100 node).
3. Output includes `nvidia-smi` (GPU name, memory) and per-epoch timing.

**Teaching moment:** compare the per-epoch time to what their laptop would do; `gpu-debug` charges vs `gpu`; mention NGC containers (PyTorch NGC image) as the reproducible alternative to modules.

**Checkpoint:** `nvidia-smi` output showing NVIDIA A100 in their job output file. *This is the wow moment — leave time for it. Fallback if the `gpu-debug` queue is saturated (check pre-session): instructor demos from the front; attendees submit and leave with a `PENDING` job + instructions to check output later — spun as “the real HPC experience: your job runs when the resource is free.”*

**Stretch — Job arrays (if at least 10 min remain, 2:12–2:15)** `labs/array.sbatch` — 5-element array sweeping a learning rate; `SLURM_ARRAY_TASK_ID`. Show `$SLURM_SUBMIT_DIR/slurm-<id>_<taskid>.out`. Frame as “one script, twenty experiments.”

### Part 3 — Wrap-up (2:15–2:30, 15 min)

| #  | Slide                         | Time  | Content                                                                                                                                                                                                                                                                          |
|----|-------------------------------|-------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 8  | What you did today            | 3 min | Recap: connected, submitted, monitored, ran CPU + GPU AI jobs. Literal checklist.                                                                                                                                                                                                |
| 9  | Your next job is the hard one | 4 min | Getting your <i>own</i> allocation: ACCESS credits -> transfer to Anvil; NAIRR Pilot allocations (automatic approval path); point to the Beginner Track 2 API session and Advanced tracks.                                                                                       |
| 10 | Rules of the road             | 3 min | Login-node etiquette; <code>debug/gpu-debug</code> for testing; charge factors ( <code>highmem = 4x</code> , <code>node-exclusive</code> <code>wholenode</code> bills 128 cores even if you use 1); where to get help ( <code>rcac-help@purdue.edu</code> , docs, office hours). |
| 11 | Q&A + resources QR            | 5 min | QR code to lab files + docs; stay for the break — Track 2 starts 2:45.                                                                                                                                                                                                           |

### Timing risk register

| Risk                                    | Mitigation                                                                              |
|-----------------------------------------|-----------------------------------------------------------------------------------------|
| SSH keys not set up (largest time sink) | Pre-provisioned accounts + OnDemand browser fallback; helpers roam the room             |
| <code>debug</code> queue busy at 1 PM   | Scripts also run fine on <code>shared</code> (default) — one-line change in the handout |
| <code>gpu-debug</code> saturation       | Front-of-room demo + submit-and-check-later framing (above)                             |

| Risk                       | Mitigation                                                                                                                           |
|----------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| 90 min is tight for 4 labs | Labs 3 and 4 are the cut line: Lab 3 can drop to instructor-demo if running >5 min behind; never cut Lab 2 (it's LO3) or the wrap-up |
| Mixed skill levels         | Checkpoint system + stretch content for fast finishers                                                                               |

## Success criteria

- at least 80% of attendees produce their own `slurm-*.out` (Lab 2 checkpoint)
- at least 60% see A100 `nvidia-smi` output from their own GPU job (Lab 4 checkpoint)
- Every attendee leaves with the handout + lab files and knows the two mandatory sbatch lines (`-A`, `-p`) by heart

## Differentiation

- **Never touched a terminal:** OnDemand GUI path runs every lab except the stretch; handout marks the GUI-equivalent steps in each lab.
- **Has a cluster account elsewhere:** “if you know Slurm, you know Anvil” — differences box in the handout (account string format, `mybalance`, `showpartitions`, debug queues).
- **Fast finishers:** stretch lab, then explore `sacct` columns / charge factors.